Formal Methods and Specification (SS 2025/26)
Lecture 10: Abstraction, Decision Procedures

Stefan Ratschan

Czech Technical University in Prague

Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Motivation

Methods already discussed need checks of verification conditions

Can, to a large extent, be done automatically.

Powerful solvers (e.g., CVC4) reason for success in recent years.

Prelude: Validity vs. Satisfiability

For example: real numbers

$$\models x + 1 \geq x$$

or, equivalently

$$\neg x + 1 \geq x \text{ is not satisfiable}$$

and, in general:

$$\models \quad \phi$$

iff

formula $\neg\phi$ is not satisfiable

counter-example to $\models \quad \phi \quad \longleftrightarrow \quad$ assignment satisfying $\neg\phi$

Prelude: Validity vs. Satisfiability

For example: real numbers

$$\models \neg x + 1 \geq x \text{ that is } x + 1 < x$$

or, equivalently

$$x + 1 \geq x \text{ is unsatisfiable}$$

and, in general:

$$\models \neg\phi$$
iff
formula ϕ is not satisfiable

assignment satisfying ϕ $\longleftrightarrow$ counter-example to $\models \neg\phi$

Example:

$Q \leftarrow \bot$
if $200 + 314x \geq 2 \wedge \text{end\_of\_the\_world} > 2073$ **then**
    $Q \leftarrow \top$
if $[\neg[\text{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q$ **then**
    explosion!

Two catastrophic execution paths:

1. skipping first **if**
2. executing first **if**

Check whether executable (see symbolic execution formulas):

$$\neg Q \wedge \neg[200 + 314x \geq 2 \wedge \text{end\_of\_the\_world} > 2073] \wedge [\neg[\text{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q$$

$$\neg Q \wedge 200 + 314x \geq 2 \wedge \text{end\_of\_the\_world} > 2073 \wedge Q_1 \wedge [\neg[\text{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q_1$$

Everything o.k., if both formulas not satisfiable (*unsat*) (demo)

Unsat Proofs

$$\models \neg\phi$$

iff

formula ϕ is not satisfiable

So: To check satisfiability, we try to prove $\neg\phi$.

We assume ϕ, and try to find a contradiction

(or a satisfying assignment)

Unsat Proof of First Formula

$Q \leftarrow \bot$
if $200 + 314x \geq 2 \wedge \mathrm{end\_of\_the\_world} > 2073$ **then**
    $Q \leftarrow \top$
if $[\neg[\mathrm{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q$ **then**
    explosion!

To prove unsatisfiability of

$$\neg Q \wedge \neg[200 + 314x \geq 2 \wedge \mathrm{end\_of\_the\_world} > 2073] \wedge$$
$$[\neg[\mathrm{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q$$

we assume this formula, which immediately results in a contradiction.

We only used Boolean variables, ignored everything else.

Unsat Proof of Second Formula

$Q \leftarrow \bot$
if $200 + 314x \geq 2 \wedge \mathrm{end\_of\_the\_world} > 2073$ **then**
    $Q \leftarrow \top$
if $[\neg[\mathrm{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q$ **then**
    explosion!

To prove unsatisfiability of

$$\neg Q \wedge 200 + 314x \geq 2 \wedge \mathrm{end\_of\_the\_world} > 2073 \wedge Q_1$$
$$[\neg[\mathrm{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q_1$$

we assume this formula, and try to arrive at a contradiction.

No contradiction from properties of Boolean variables alone.

Unsat Proof of Second Formula

$$\neg Q \wedge 200 + 314x \geq 2 \wedge \text{end\_of\_the\_world} > 2073 \wedge Q_1 \wedge$$
$$[\neg[\text{end\_of\_the\_world} > 2073] \vee 200 + 314x < 2] \wedge Q_1$$

We ignore details, but reflect syntactic equality of atomic formulas

$$\neg Q \wedge S \wedge R \wedge Q_1 \wedge [\neg R \vee T] \wedge Q_1$$

leads to a contradiction?

satisfiable, satisfied by

$$\{Q \mapsto \bot, S \mapsto \top, R \mapsto \top, Q_1 \mapsto \top, T \mapsto \top\}$$

$200 + 314x \geq 2$, $\text{end\_of\_the\_world} > 2073$, $200 + 314x < 2$

$$\neg Q \wedge S \wedge R \wedge Q_1 \wedge [\neg R \vee \neg S] \wedge Q_1$$

Unsat Proof of Second Formula

Assume $\neg Q \wedge S \wedge R \wedge Q_1 \wedge [\neg R \vee \neg S] \wedge Q_1$

$\neg Q, S, R, Q_1, [\neg R \vee \neg S], Q_1$

Case 1: $\neg Q, S, R, Q_1, \neg R, Q_1$ contradiction

Case 2: $\neg Q, S, R, Q_1, \neg S, Q_1$ contradiction

Summary

We started by ignoring details, and incrementally used more and more properties:

1. properties of Boolean variables
2. syntactic equality of atomic formulas
3. $<$ is the negation of $\geq$

Ignoring details that are not relevant for the given problem:

abstraction

If the abstraction is unsat then original formula is unsat,
but not the other way round

If the abstraction is satisfiable, and we want to prove unsatisfiability,
we have to add additional properties

The same holds analogically, if we want to prove that a formula holds

Program Abstraction

Application of same principle directly to programs

$Q \leftarrow \bot$
if $S \wedge R$ **then**
    $Q \leftarrow \top$
if $[\neg R \vee T] \wedge Q$ **then**
    explosion!

Every execution of original program, has a
  corresponding execution of the abstracted program
    but not the other way round

For example, $\{S \mapsto \top, R \mapsto \top, T \mapsto \top\}$ leads to explosion.

etc.

Decision Procedures

algorithms behind SMT solvers

specific logical theories (e.g., arrays, integers, real number, ...)

SMT solvers combine many decision procedures

If we do not want to make the difference: *solvers*

Proving vs. Deciding Satisfiability

Solvers usually do not prove, but check satisfiability

Reason:

- Separation into assumptions and things to prove
  is benefitial for human intuition.
- Working with assumptions only
  suits algorithms (uniform handling of formulas).

Now, solvers for various levels of abstraction:

- propositional logic
- equality
- arrays

Solvers for Propositional Logic

SAT: "satisfiability" (splnitelnost)

- Input: propositional formula, usually in conjunctive normal form
- Output:
  - satisfying assignment, if it exists,
  - `unsat`, if the formula is not satisfiable.

Example: $[\neg P \vee Q \vee R] \wedge [\neg Q \vee R] \wedge [\neg Q \vee \neg R] \wedge [P \vee Q]$

NP-hard problem, but huge efficiency improvements in recent years

Today's solvers can solve huge formulas

Further Example

if x=1 **then**
    $y \leftarrow 1$
    if adfxg7(x) $\neq$ adfxg7(y) **then**
        @ $\bot$

$$x = 1 \wedge y = 1 \wedge \text{adfxg7}(x) \neq \text{adfxg7}(y) \quad \text{satisfiable?}$$

Boolean abstraction?

$$P \wedge Q \wedge R$$

satisfied by

$$\{P \mapsto \top, Q \mapsto \top, R \mapsto \top\}$$

But is it possible that $x = 1$, $y = 1$, and adfxg7(x) $\neq$ adfxg7(y)?

If *adfxg7* is a mathematical function, certainly not!

Proof

Assume $x = 1$, $y = 1$, and $\mathrm{adfxg7}(x) \neq \mathrm{adfxg7}(y)$.

Then $\mathrm{adfxg7}(1) \neq \mathrm{adfxg7}(1)$

contradiction with law of reflexivity

Argument uses knowledge about equality,
but abstracts from the precise behavior of 1 and adfxg7.

Would have worked equally well with

$$a = c \wedge b = c \wedge f(a) \neq f(b)$$

Equality

What do we know about $=$?

- $\forall x \,.\, x = x$ (reflexivity)
- $\forall x, y \,.\, x = y \Rightarrow y = x$ (symmetry)
- $\forall x, y, z \,.\, [x = y \wedge y = z] \Rightarrow x = z$ (transitivity)
- for every function symbol f of arity n,

$$\forall x_1, \ldots, x_n, y_1, \ldots y_n \,. \\ [x_1 = y_1, \ldots, x_n = y_n] \Rightarrow f(x_1, \ldots, x_n) = f(y_1, \ldots, y_n)$$

first three axioms: equivalence relation

all axioms: congruence relation

result: *theory of free function symbols*

Congruence Closure Algorithm [Nelson and Oppen, 1980]

Assume axioms + given formula, look for contradiction.

Example [Bradley and Manna, 2007]: Checking satisfiability of

$$f(a, b) = a \wedge f(f(a, b), b) \neq a$$

Deduce equalities using:

1. equivalence axioms:
   - $\forall x \,.\, x = x$ (reflexivity)
   - $\forall x, y \,.\, x = y \Rightarrow y = x$ (symmetry)
   - $\forall x, y, z \,.\, [x = y \wedge y = z] \Rightarrow x = z$ (transitivity)
2. equalities in formula
3. congruence axioms

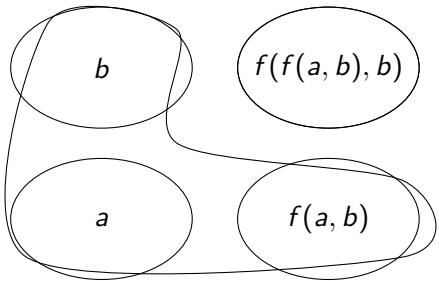
$$\forall x_1, \ldots, x_n, y_1, \ldots y_n \,.$$
$$[x_1 = y_1, \ldots, x_n = y_n] \Rightarrow f(x_1, \ldots, x_n) = f(y_1, \ldots, y_n)$$

until contradiction between deduced equality and disequality.

Congruence Closure: Using Equivalence Axioms

basic idea: any equivalence relation defines equivalence classes

. . . on the set of sub-terms of the given formula



every equivalence class represents all equalities between its members

initial equivalence classes: every sub-term is in a separate class:

$$\{\{a\},\{b\},\{f(a,b)\},\{f(f(a,b),b)\}\}$$

Congruence Closure: Using Equalities, Congruence Axioms

initial equivalence classes: every sub-term is in a separate class:

$$\{\{a\}, \{b\}, \{f(a,b)\}, \{f(f(a,b),b)\}\}$$

from $f(a,b) = a$:

$$\{\{a, f(a,b)\}, \{b\}, \{f(f(a,b),b)\}\}$$

congruence axiom:

$$\forall x_1, \ldots, x_n, y_1, \ldots y_n \,.\, [x_1 = y_1, \ldots, x_n = y_n] \Rightarrow f(x_1, \ldots, x_n) = f(y_1, \ldots, y_n)$$

congruence propagation: from $\{a, f(a,b)\}$: $f(f(a,b),b) = f(a,b)$, merge class:

$$\{\{a, f(a,b), f(f(a,b),b)\}, \{b\}\}$$

contradiction with $f(f(a,b),b) \neq f(a,b)$, unsat

Congruence Closure Algorithm [Nelson and Oppen, 1980]

- Input: formula $s_1 = t_1 \wedge \cdots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \cdots \wedge s_n \neq t_n$
- Output: `unsatisfiable`, `satisfiable`
  (in the theory of free function symbols)

Let τ be the set of sub-terms of

$$s_1 = t_1 \wedge \cdots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \cdots \wedge s_n \neq t_n$$

$\sim \leftarrow$ equivalence relation on τ s.t. for all $u, v \in \tau$, $u \neq v$ implies $u \not\sim v$

while there is $i \in \{1 \ldots m\}$ s.t. $s_i \not\sim t_i$ **do**

// loop invariant: if $u \sim v$ then $u = v$ follows from axioms+formula

  merge equivalence classes of s_i and t_i in $\sim$

  while there is $p, q \in \tau$, $u, v \in \tau$ s.t.

$$p \sim q,\ [u[p \leftarrow q] = v \text{ or } u[q \leftarrow p] = v],\ u \not\sim v$$ **do**

    merge equivalence classes of u and v in $\sim$

if there is an $i \in \{m+1, \ldots, n\}$ s.t. $s_i \sim t_i$ **then**

  return `unsatisfiable`

else

  return `satisfiable`

General Formulas: Disjunctions, Quantifiers

Example:

$$a \neq a \vee [a = b \wedge f(a) \neq f(b)]$$

satisfiable?

- $a \neq a$?: unsat
- $a = b \wedge f(a) \neq f(b)$?: unsat

Hence, the original formula is unsat

In general:

A formula $\phi_1 \vee \cdots \vee \phi_n$ is satisfiable iff
one of the formulas $\phi_1, \ldots, \phi_n$ is satisfiable.

Hence: formulas with conjunctions and disjunctions: disjunctive normal form

SMT solvers: combination with SAT solvers, input in conjunctive normal form

General case with quantifiers: undecidable

Using the Theory of Free Function Symbols

Example: proving unsatisfiability of

$$a[x] = 0 \wedge x = y \wedge a[y] = 1$$

abstracted formula:

$$f(a, x) = c \wedge x = y \wedge f(a, y) = d$$

sub-terms: $\{a, c, d, x, y, f(a, x), f(a, y)\}$

result: satisfiable, equivalence classes: $\{a\}, \{x, y\}, \{c, d, f(a, x), f(a, y)\}$

refinement of abstraction:

$$f(a, x) = c \wedge x = y \wedge f(a, y) = d \wedge c \neq d$$

unsat

Using the Theory of Free Function Symbols

Further example: pointers (abstract $*x$ to $f(x)$)

Application to data structures: abstract from detailed properties,
and try a proof just using equality i.e., in the theory of free function symbols

Sometimes we can add additional knowledge (e.g., $0 \neq 1$, commutativity)

Arrays

$write(a, i, 7)[i] \neq 7$ satisfiable? abstract to $r(w(a, i, 7), i) \neq 7$?

Axioms

Read-same (RS): $\forall a, v, i, j \,.\, i = j \Rightarrow \mathsf{write}(a, i, v)[j] = v$

Read-different (RD): $\forall a, v, i, j \,.\, i \neq j \Rightarrow \mathsf{write}(a, i, v)[j] = a[j]$

...

Example:

$$i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \mathsf{write}(\mathsf{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j]$$

Is this satisfiable? Add to assumptions, try to find contradiction:

$i_2 = j$? If yes, RS, if no, RD.

Try both cases, after adding the case distinction $i_2 = j \vee i_2 \neq j$

Case $i_2 = j$

$$i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j]$$

Due to RS $\text{write}(\text{write}(a, i_1, v_1), i_2, v_2)[j] = v_2$

but $i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2$ is not satisfiable
and hence the whole formula in the case $i_2 = j$.

Case $i_2 \neq j$

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \mathsf{write}(\mathsf{write}(a, i_1, v_1), i_2, v_2)[j] \neq a[j]$$

Due to RD $\mathsf{write}(\mathsf{write}(a, i_1, v_1), i_2, v_2)[j] = \mathsf{write}(a, i_1, v_1)[j]$

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge \mathsf{write}(a, i_1, v_1)[j] \neq a[j]$$

Due to RS $\mathsf{write}(a, i_1, v_1)[j] = v_1$

$$i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge v_1 \neq a[j]$$

$a[j] = v_1 \wedge v_1 \neq a[j]$ is not satisfiable,
and hence the whole formula in the case $i_2 \neq j$.

The original formula is neither satisfiable in the case $i_2 = j$,
nor in the case $i_2 \neq j$,
and hence the whole formula is not satisfiable.

Decision Procedure for the Theory of Arrays

- Input: a conjunctive quantifier-free formula ϕ
- Output: `unsatisfiable`, `satisfiable`
  (in the theory of arrays (with equality))

function $\mathrm{sat}_{\mathcal{A}}(\phi)$
if ϕ is satisfiable in the theory of free function symbols **then**
  choose a term of the form $\mathrm{write}(a, i, v)[j]$ in ϕ
  if ϕ does not contain such a term **then**
    return `satisfiable`
  if $\mathrm{sat}_{\mathcal{A}}(F[\mathrm{write}(a, i, v)[j] \leftarrow v] \wedge i = j)$ **then**
    return `satisfiable`
  if $\mathrm{sat}_{\mathcal{A}}(F[\mathrm{write}(a, i, v)[j] \leftarrow a[j]] \wedge i \neq j)$ **then**
    return `satisfiable`
return `unsatisfiable`

Generalization, Further Theories

Quantifiers, axiom of equality of arrays

[Bradley et al., 2006]

[Bradley and Manna, 2007]

General case: undecidable

Theory	Full	QFF
equality	no	yes
Peano arithmetic (i.e., $\mathbb{N}$ without $\times$)	no	no
Presburger arithmetic (i.e., $\mathbb{N}$ without $\times$)	yes	yes
real numbers (with $\times$)	yes	yes
real numbers (with sin)	no	no
recursive data structure (e.g., lists)	no	yes
arrays	no	yes

(QFF=quantifier free formulas)

Combination of theories [Nelson and Oppen, 1979]

Witnesses and Their Usage

Decision procedures sometimes also return satisfying assignments
(*a witness for satisfiability*, counter-example to validity of the negation)

For example, in the theory of integers:

Input: $x + y \geq 0 \wedge x - 2y \leq 1$
Output: sat, $\{x \mapsto -1, y \mapsto 1\}$

This allows automatic execution of I/O specifications: (demo)

Spec:

- Input: $I(x)$
- Output: y s.t. $O(x, y)$

For given input a, corresponding output:
witness of satisfiability of $x = a \wedge I(x) \wedge O(x, y)$

How to ensure efficiency of automatic execution of specifications?

Real Numbers

Real numbers:

- conjunctions of linear equations: Gaussian elimination
- conjunctions of linear equations and inequalities: simplex algorithm

Sometimes it is even possible to compute something in undecidable cases

`http://rsolver.sourceforge.net`

Summary

Levels of abstraction:

- Boolean: SAT solvers
- equality between function symbols: congruence closure
- individual theories: decision procedures for arrays, real numbers etc.

In theory (decidability) level of individual theories would be enough

In practice, choice of level very important.

SAT modulo theory (SMT): tight integration:

- SAT-solver
- solver for individual theories

Examples: CVC4, z3, openSMT

Open Research Questions

Only 20-30 years ago: paper and pencil.

Today: speed, speed, speed, speed!

`https://smt-comp.github.io/`

Conclusion

Basis for automatization of software verification: decision procedures

If one chooses the right level of abstraction,
it is possible to prove properties of extremely complex systems.

Even the source code of a program is an abstraction of the resulting system:

The compiler may be incorrect, the operating system, or
we might even have the end of the world.

Literature I

Aaron Bradley and Zohar Manna. *The Calculus of Computation*. Springer, 2007.

Aaron Bradley, Zohar Manna, and Henny Sipma. What's decidable about arrays? In *Verification, Model Checking, and Abstract Interpretation*, volume 3855 of *Lecture Notes in Computer Science*, pages 427–442. Springer Berlin / Heidelberg, 2006.

Greg Nelson and Derek C. Oppen. Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst.*, 1(2):245–257, 1979. ISSN 0164-0925.

Greg Nelson and Derek C. Oppen. Fast decision procedures based on congruence closure. *J. ACM*, 27(2):356–364, April 1980.